VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors-Microcontrollers

Lab 2

Advisor: Phan Văn Sỹ
Class: CC04

Students: Nguyễn Văn Thắng – 2353123

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Exercise 1 **2**

2 Exercise 2 **3**

3 Exercise 3 **3**

4 Exercise 4 **4**

5 Exercise 5 **5**

6 Exercise 6 **6**

7 Exercise 7 **6**

8 Exercise 8 **7**

9 Exercise 9 **8**

10 Exercise 10 **9**

List of Figures

1 Schematic/FSM 2
2 The schematic of traffic light system 3
3 picture of ioc system 3
4 picture of system tree 4

1 Exercise 1

Sketch an FSM

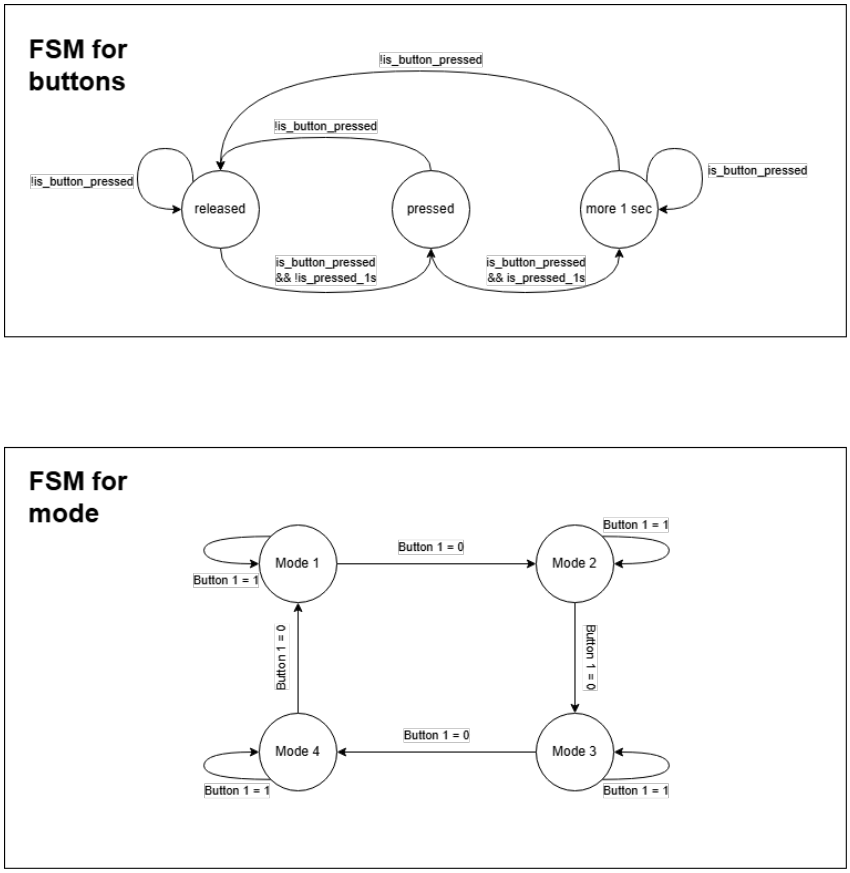


Figure 1: Schematic/FSM

To solve the problem efficiently, the system utilizes two concurrent Finite State Machines driven by a 10ms hardware timer, ensuring real-time multitasking without blocking delays:

- **Button FSM:** Handles mechanical debouncing and long-press detection (transitions between RELEASED, PRESSED, and PRESSED_MORE_THAN_1S) to provide stable user inputs.
- Mode FSM: Manages the overall system states. Mode 1 executes the normal traffic light cycle. Modes 2, 3, and 4 allow the user to configure the durations for the Red, Amber, and Green LEDs, respectively.

This dual-FSM architecture seamlessly integrates 7-segment LED multiplexing, button scanning, and traffic state switching.

2 Exercise 2

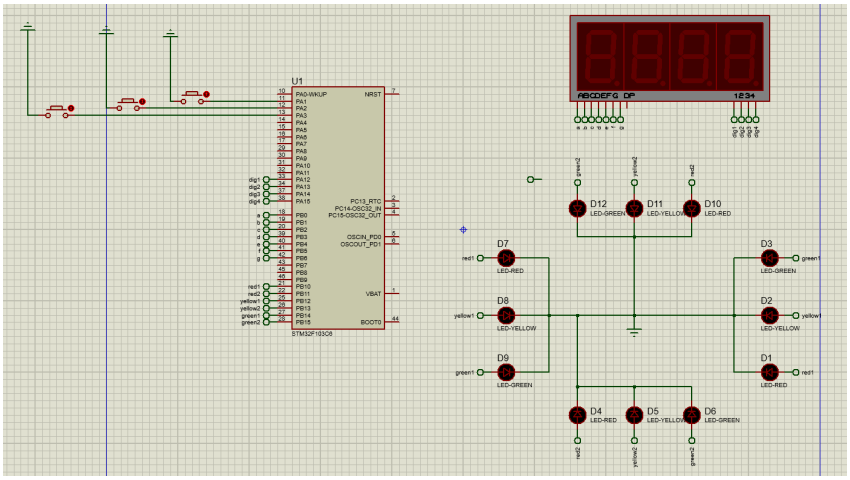


Figure 2: The schematic of traffic light system

3 Exercise 3

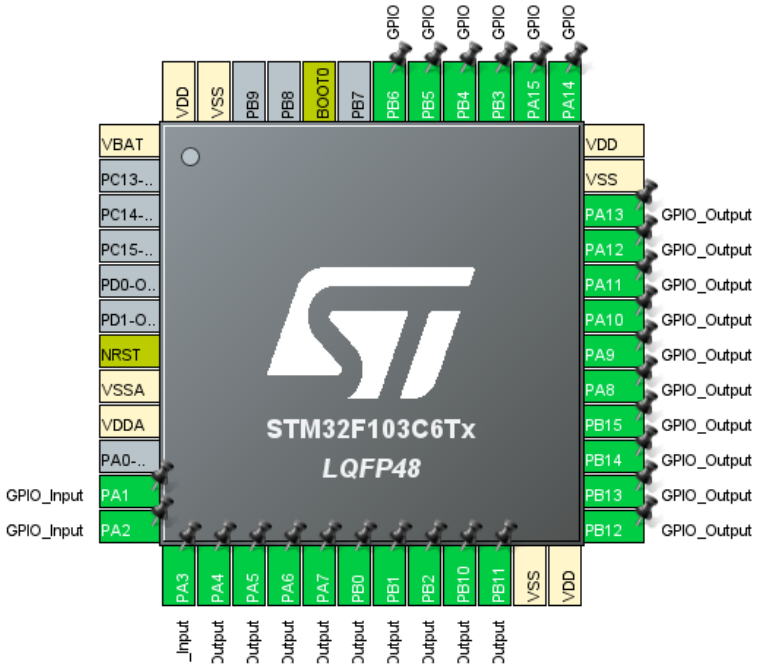


Figure 3: picture of ioc system

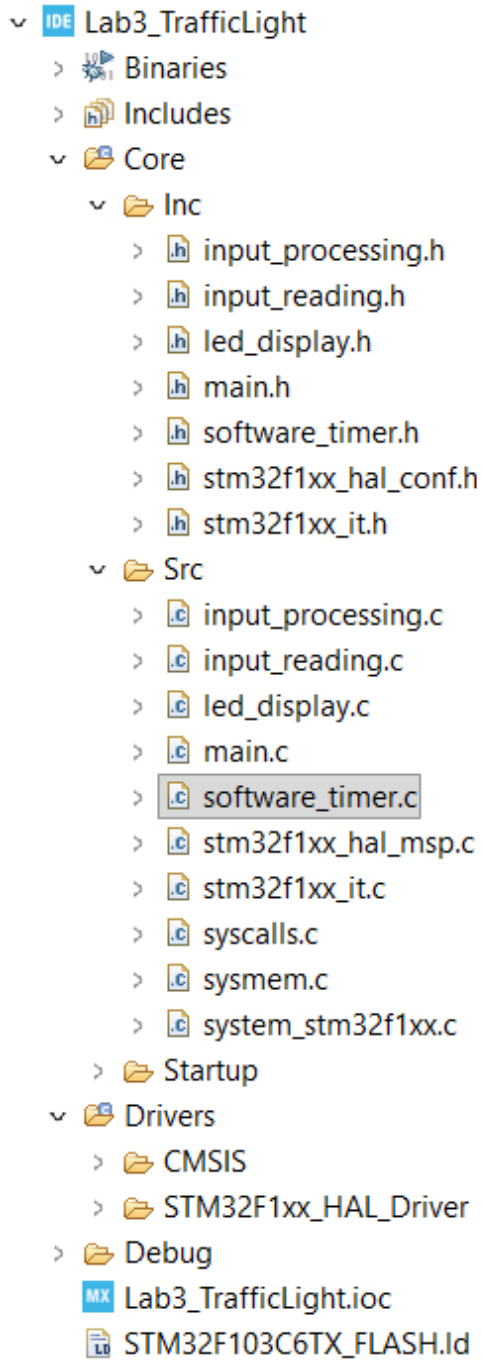


Figure 4: picture of system tree

- GPIO_Output:PA4-PA14; PB0-PB12
- GPIO_Input: PA1-PA3

4 Exercise 4

Listing 1: Modified Timer Setup

```
int TIMER_CYCLE = 10;

```

```
void setTimer(int index, int duration) {
    timer_counter[index] = duration / TIMER_CYCLE;
    timer_flag[index] = 0;
}
```

5 Exercise 5

1. Button Debouncing Code

Listing 2: Button debouncing logic in input_reading.c

```
void button_reading(void) {
    for (int i = 0; i < NO_OF_BUTTONS; i++) {
        debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
        debounceButtonBuffer1[i] = debounceButtonBuffer0[i];
        debounceButtonBuffer0[i] = HAL_GPIO_ReadPin(GPIOA,
           button_pins[i]);

        if ((debounceButtonBuffer2[i] == debounceButtonBuffer1[i
           ]) &&
            (debounceButtonBuffer1[i] == debounceButtonBuffer0[i
               ])) {

            if (buttonBuffer[i] != debounceButtonBuffer2[i]) {
                buttonBuffer[i] = debounceButtonBuffer2[i];
                if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
                    button_flag[i] = 1;
                }
            }
        }
    }
}
```

2. Increasing Mode upon Button 1 Press

Listing 3: Handling Mode transition in input_processing.c

```
if (is_button_pressed(0)) {
    if (button1_flag == 0) {
        button1_flag = 1;
        system_mode++;
        if (system_mode > 4) {
            system_mode = 1;
        } else {
            clearAllTrafficLights();
            setTimer(1, 250);
        }
    }
} else {
    button1_flag = 0;
```

```
}
```

6 Exercise 6

The system utilizes a switch-case statement within the main FSM to manage the behaviors of different configuration modes.

1. Displaying Modes on Seven-Segment LEDs

A helper function update_led_buffer(val1, val2) is used to split the values and push them into the 4-digit 7-segment display array. During configuration modes, the first two digits display the temporary time value (temp_value), while the last two digits display the current mode (2, 3, or 4).

2. Blinking LEDs at 2Hz

To create the 2Hz blinking effect without using blocking delays, a dedicated software timer (`timer_flag[1]`) is set to 250ms. When the timer flag is triggered, the `HAL_GPIO_TogglePin` function toggles the corresponding traffic light pins based on the active mode.

Listing 4: Display and Blinking logic based on Mode

```
switch (system_mode) {
    case 2:
        update_led_buffer(temp_value, 2);
        if (timer_flag[1] == 1) {
            setTimer(1, 250);
            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_10 | GPIO_PIN_11);
        }
        break;

    case 3:
        update_led_buffer(temp_value, 3);
        if (timer_flag[1] == 1) {
            setTimer(1, 250);
            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_12 | GPIO_PIN_13);
        }
        break;

    case 4:
        update_led_buffer(temp_value, 4);
        if (timer_flag[1] == 1) {
            setTimer(1, 250);
            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_14 | GPIO_PIN_15);
        }
        break;
}
```

7 Exercise 7

1. Increasing the Time Duration (Button 2)

When Button 2 is pressed, a temporary variable (temp_value) is incremented by 1. To

ensure the 2-digit 7-segment display works correctly, if the value exceeds 99, it wraps around to 1. The debouncing flag ensures that one physical press corresponds to exactly one increment.

2. Setting the Value (Button 3)

When Button 3 is pressed, the system takes the current temp_value and assigns it to the global red_time variable, officially updating the duration for the Red LEDs in the normal operation cycle.

Listing 5: Logic for modifying Red LED duration in Mode 2

```
case 2:
    if (is_button_pressed(1)) {
        if (button2_flag == 0) {
            button2_flag = 1;
            temp_value++;
            if (temp_value > 99) {
                temp_value = 1;
            }
        }
    } else {
        button2_flag = 0;
    }

    if (is_button_pressed(2)) {
        if (button3_flag == 0) {
            button3_flag = 1;
            red_time = temp_value;
        }
    } else {
        button3_flag = 0;
    }
    break;
```

8 Exercise 8

Similar to Mode 2, Mode 3 allows the user to adjust the time duration specifically for the Amber LEDs. The logic remains consistent, utilizing the same debounced buttons to ensure reliable user input.

1. Increasing the Time Duration (Button 2)

Pressing Button 2 increments the temp_value. The value is constrained within a two-digit limit (1 to 99) to fit the 7-segment display, wrapping back to 1 if it exceeds the maximum limit.

2. Setting the Value (Button 3)

Pressing Button 3 confirms the selection, saving the current temp_value to the global amber_time variable. This updated duration will automatically take effect when the system returns to the normal traffic light cycle.

Listing 6: Logic for modifying Amber LED duration in Mode 3

```
case 3:
    if (is_button_pressed(1)) {
```

```
        if (button2_flag == 0) {
            button2_flag = 1;
            temp_value++;
            if (temp_value > 99) {
                temp_value = 1;
            }
        }
    } else {
        button2_flag = 0;
    }

    if (is_button_pressed(2)) {
        if (button3_flag == 0) {
            button3_flag = 1;
            amber_time = temp_value;
        }
    } else {
        button3_flag = 0;
    }
    break;
```

9 Exercise 9

Mode 4 completes the configuration cycle by allowing the user to adjust the time duration for the Green LEDs. The logic is identical to the previous modes, utilizing the second and third buttons.

1. Increasing the Time Duration (Button 2)

Pressing Button 2 increments the temp_value. The value is constrained within a two-digit limit (1 to 99) for the 7-segment display, wrapping back to 1 if it exceeds the maximum limit.

2. Setting the Value (Button 3)

Pressing Button 3 confirms the selection, saving the current temp_value to the global green_time variable. This updated duration will automatically take effect when the system returns to Mode 1 (normal operation).

Listing 7: Logic for modifying Green LED duration in Mode 4

```
case 4:
    if (is_button_pressed(1)) {
        if (button2_flag == 0) {
            button2_flag = 1;
            temp_value++;
            if (temp_value > 99) {
                temp_value = 1;
            }
        }
    } else {
        button2_flag = 0;
    }

```

```
14      if (is_button_pressed(2)) {
15          if (button3_flag == 0) {
16              button3_flag = 1;
17              green_time = temp_value;
18          }
19      } else {
20          button3_flag = 0;
21      }
22      break;
```

10 Exercise 10

You can find the complete STM32CubeIDE project and Proteus schematic on my GitHub repository: `https://github.com/thanglun0510/VXL-Traffic-Light-Lab3`